

Práctica final

**Estructura de
computadores
2020-2021 - G01**

Jaume Adrover

43235882E - jaume.adover3@estudiant.uib.cat

Joan Balaguer

43219174N - joan.balaguer2@estudiant.uib.cat

Índice

1. Introducción	3
2. Fase de FETCH	3
3. Fase de DECODIFICACIÓN	4
4. Fase de EJECUCIÓN	5
5. Juego de pruebas	9
6. Conclusiones	12
7. Código fuente del programa	13

1. Introducción

En esta práctica se nos pide que llevemos a cabo la implementación de una CPU emulada dentro de la memoria del 68K. Esta, ha sido bendecida con el nombre de CDB (Computador Didáctico Balear) y nuestro objetivo es que esta, simule el ciclo de una CPU con arquitectura Von Neuman. Por lo tanto, nuestro programa se puede dividir en 3 grandes bloques: el de la fase de FETCH (donde meteremos en el registro EIR la siguiente instrucción a realizar y dejaremos el EPC apuntando a la dirección de la siguiente instrucción a realizar), el de la fase de DECODIFICACIÓN (donde tenemos como objetivo principal determinar qué tipo de instrucción se está llevando a cabo) y el de la fase de EJECUCIÓN (en la cual, dependiendo del resultado de la fase de decodificación, se llevará a cabo la ejecución de una u otra instrucción según el repertorio de instrucciones dado en el enunciado de esta práctica). Este ciclo con 3 fases, lo realizaremos hasta que detectemos la instrucción `HLT`, la cual parará la ejecución del programa.

Tal y como se nos indica en el enunciado, este emulador tiene: 5 registros de propósito general (`ER1`, `ER2`, `ER3`, `ER4`, `ER5`), 2 registros que servirán de interfaz con la memoria (`ET6` y `ET7`), el registro `EIR`, donde guardaremos la instrucción que vamos a realizar en un instante de tiempo determinado; el vector `EPROG`, donde estarán almacenadas las instrucciones del programa que queremos llevar a cabo; el registro `EPC`, que contendrá la dirección de la siguiente instrucción a realizar y por último el registro `ESR`, el cual contendrá en sus 3 bits menos significativos los valores de los flags de la máquina emulada distribuido de la siguiente forma: `NCZ`. Además, hemos añadido una etiqueta para identificar el valor 0. A esta la hemos llamado `X` y su utilidad se verá en la implementación de la fase de FETCH.

Toda esta información queda claramente sintetizada en la cabecera del programa.

2. Fase de FETCH

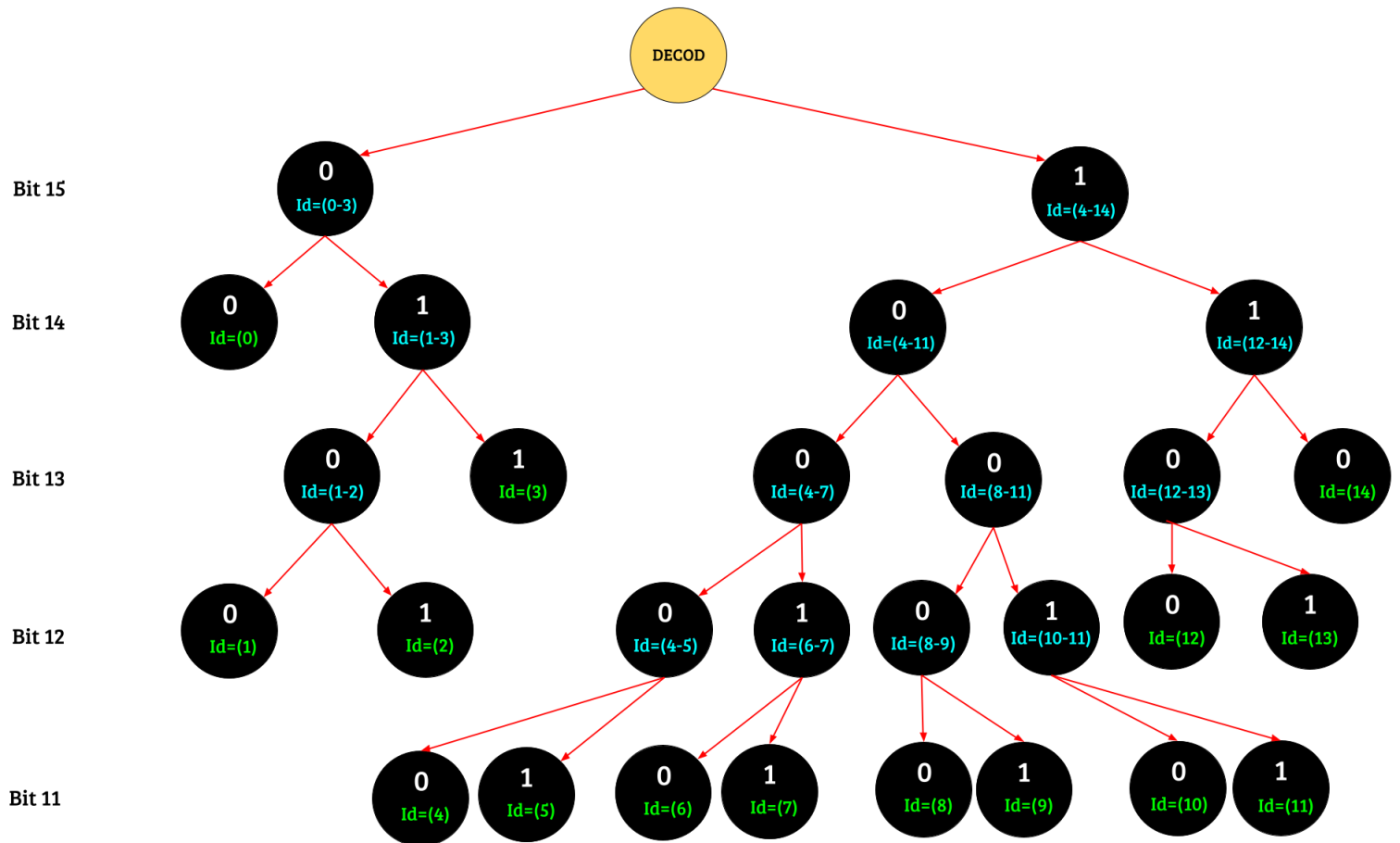
En esta fase, nuestro principal objetivo es que, al final de la misma, el registro `EIR` contenga el código de la instrucción que realizaremos en el ciclo en el que nos encontremos y que el registro `EPC` contenga la dirección de la siguiente instrucción a realizar. Para que dichas condiciones se cumplieran, primero de todo hemos inicializado a 0 el valor del `EPC` (esto solo lo realizaremos en el primer ciclo) y cargamos la dirección de memoria del vector `EPROG` en el registro de direcciones `A0` para mayor comodidad a la hora de pasar de una instrucción del programa a otra. Ya iniciada la fase de FETCH, usaremos el registro del 68K `D0` para almacenar el valor actual del `EPC` y este, lo multiplicaremos por 2 (ya que debemos hacer la

conversión de las direcciones emuladas a las direcciones reales del 68K y usaremos el contenido del `EPC · 2` como índice para ir saltando de una instrucción de `EPROG` a otra). Una vez tenemos el índice adecuado introducido en `D0`, nos aprovecharemos de la capacidad que tiene modo de direccionamiento indexado completo para poder calcular la dirección efectiva con el contenido de un registro de datos. Lo usaremos para mover el contenido de la posición de memoria que se va a calcular mediante la suma de la dirección contenida en el registro `A0` (dirección donde se encuentra `EPROG`)+el contenido del registro `D0` (registro que se va a ir modificando ciclo tras ciclo)+ `X` (etiqueta para identificar el 0, que realmente no hace nada, pero queda mejor que poner un 0). Una vez calculada por el indexado completo, la dirección efectiva nos estará indicando la dirección en memoria de la instrucción de `EPROG` que vamos a realizar en este ciclo. Por lo tanto, moveremos el contenido de dicha dirección a `EIR`. Por último, incrementaremos en 1 el valor del `EPC`, para dejarlo preparado para el siguiente ciclo.

3. Fase de DECODIFICACIÓN

En este punto del programa, tenemos en el registro `EIR` la instrucción que la máquina debe realizar. Por lo tanto, ya estamos en disposición de decodificar dicha instrucción. Para ello, llevamos a cabo la subrutina de librería `DECOD`, la cual nos identificará la instrucción que le demos por parámetro e introducirá en la pila del 68K un número del 0-14. Este, será nuestro identificador para poder saber qué instrucción realizar. Pero antes de saltar a la subrutina, hay que hacer un par de pasos previos. Primero de todo, reservamos un word en la pila para poder almacenar, ya dentro de la subrutina, el identificador de la instrucción a realizar (resultado de la decodificación). El segundo paso previo al salto, es hacer un `PUSH` (meter en la pila) del código de la instrucción contenido en el registro `EIR`, el cual la subrutina nos va a decodificar. Una vez hecho esto, estamos en condiciones de saltar a la subrutina `DECOD`. Esta, va a dejar en la posición de la pila reservada para el resultado, el número del identificador del tipo de instrucción que vamos a llevar a cabo. Con esta información contenida en la pila, al volver de la subrutina, solo tendremos que hacer un `POP` de dicha información al registro `D1` con el cual vamos a saber qué instrucción realizar en la fase de ejecución.

Árbol de decodificación con el cual sabemos qué identificador debemos introducir en la pila:



4. Fase de EJECUCIÓN

Finalmente llegamos a la última parte del ciclo, donde llevaremos a cabo la instrucción que nos indique el identificador contenido en el registro D1. Para cada tipo de instrucción llevaremos a cabo las indicaciones que el repertorio de instrucciones del enunciado de la práctica nos ordena qué tiene que hacer cada instrucción. A continuación, se adjunta una tabla con todas las subrutinas que se han llevado a cabo para la realización de todas las instrucciones de la fase de ejecución:

Nombre de la subrutina	Libreria o Usuario	Interfaces de entrada	Interfaces de salida
DECOD	Libreria	Reservamos un word en la pila para almacenar el identificador resultante de la decodificación y también almacenamos el valor actual de <code>EIR</code>	Almacenamos el valor del identificador en la pila, para poder extraerlo e insertarlo en el registro <code>D1</code> una vez volvamos de la subrutina
REGISTERa	Usuario	El contenido del registro <code>EIR</code> será introducido en <code>D4</code> para poder analizar los bits 5, 6 y 7 que nos indican qué registro aaa nos está indicando la instrucción contenida en <code>EIR</code> .	Movemos el contenido de cada registro del emulador, según el valor de los bits aaa de <code>EIR</code> , al registro <code>D5</code> , donde siempre tendremos almacenado el contenido del operando fuente de la instrucción.
REGISTERb	Usuario	El contenido del registro <code>EIR</code> será introducido en <code>D4</code> para poder analizar los bits 0, 1 y 2 que nos indican qué registro bbb nos está indicando la instrucción contenida en <code>EIR</code> .	Movemos el contenido de cada registro del emulador, según el valor de los bits bbb de <code>EIR</code> , al registro <code>D6</code> , donde siempre tendremos almacenado el contenido del operando destino de la instrucción. Además, almacenamos la dirección de memoria de cada registro en el registro <code>A3</code> , para poder (fuera de la subrutina) almacenar el valor del resultado

FLAGZ	Usuario	EL contenido del registro ESR (que almacenaremos en el registro D3) y el contenido del registro D6 (donde siempre tendremos almacenado el resultado de la operación realizada antes de llamar a la subrutina).	Modificación del bit 0 del registro ESR , dependiendo de si el resultado es igual a 0 o no.
FLAGN	Usuario	EL contenido del registro ESR (que almacenaremos en el registro D3) y el contenido del registro D6 (donde siempre tendremos almacenado el resultado de la operación realizada antes de llamar a la subrutina)	Modificación del bit 2 del registro ESR , dependiendo de si el resultado es negativo o no.
FLAGC	Usuario	EL contenido del registro ESR (que almacenaremos en el registro D3 antes de llamar a la subrutina) y el contenido del registro D6 (donde siempre tendremos almacenado el resultado de la operación realizada antes de llamar a la subrutina).	Modificación del bit 2 del registro ESR , dependiendo de si el flag N del 68K vale 1 o 0.

En esta segunda tabla, se muestra la función que realizan cada uno de los registros del 68K:

REGISTRO	FINALIDAD	REGISTRO DIRECCIONES	FINALIDAD
D0	Almacenar el valor del EPC y multiplicar por 2 para realizar el salto correspondiente.	A0	Ubicar la dirección del EPROG mediante un LEA.L, que nos servirá para medir la posición de la memoria para movernos dentro del programa emulado.
D1	Almacenar el valor de la decodificación obtenido dentro de DECOD.	A1	Registro que ya viene por defecto con la finalidad de llamar un jumplist y recorrer las instrucciones a ser ejecutadas.
D2		A2	
D3	Almacenar el valor del status register (ESR), que será modificado con BTST.L	A3	Almacenar el valor del resultado a un determinado registro que tendrá la dirección en A3.
D4	Almacenar el valor de EIR y obtener el valor de los registros o atributos correspondientes a la instrucción.	A4	
D5	Registro utilizado para almacenar el contenido del Registro 'aaa' de EIR.	A5	
D6	Registro utilizado para almacenar el contenido del Registro 'bbb' de EIR.	A6	
D7		A7	

: No se utiliza el registro.

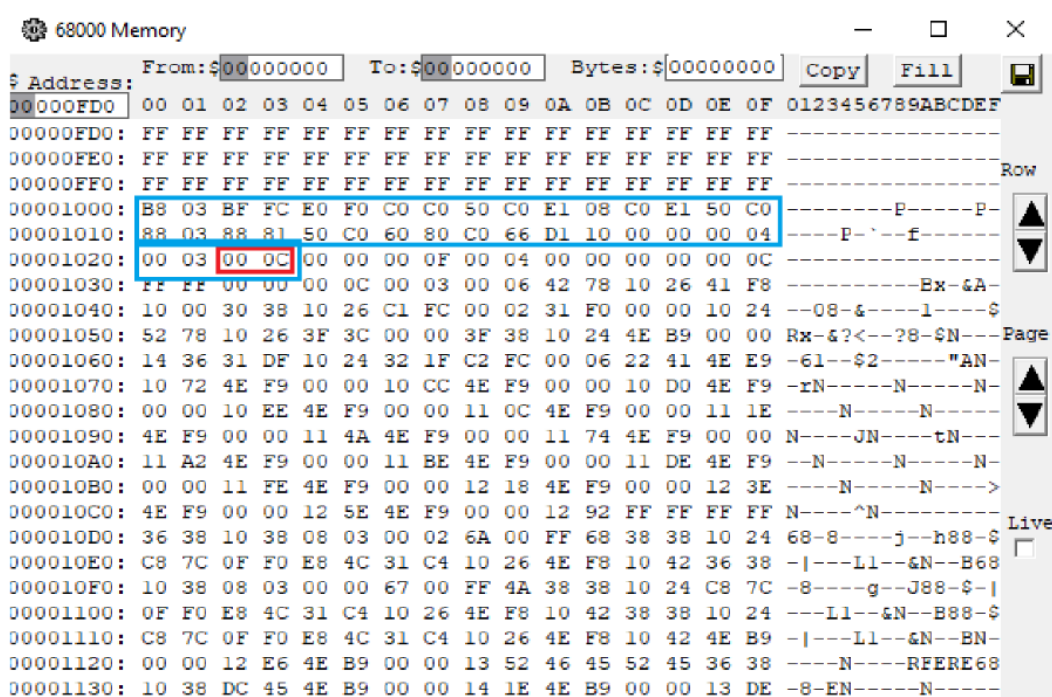
5. Juego de pruebas

✓ Ejemplo de programa número 1 para la CDB.

Etiqueta	Ensamblador con etiquetas	Dirección @CDB	Ensamblador sin etiquetas	Instrucciones codificadas	Hex
	SET #0,R3	0:	SET 0,R3	1011100000000011	B803
	SET #-1,R4	1:	SET -1,R4	1011111111111100	BFFC
	LOA A,T6	2:	LOA 15,T6	1110000011110000	E0F0
	MOV T6,R0	3:	MOV T6,R0	1100000011000000	C0C0
	JMZ END	4:	JMZ 12	0101000011000000	50C0
	LOA B,T7	5:	LOA 16,T7	1110000100001000	E108
	MOV T7,R1	6:	MOV T7,R1	1100000011100001	C0E1
	JMZ END	7:	JMZ 12	0101000011000000	50C0
LOOP:	ADD R0,R3	8:	ADD R0,R3	1000100000000011	8803
	ADD R4,R1	9:	ADD R4,R1	1000100010000001	8881
	JMZ END	10:	JMZ 12	0101000011000000	50C0
	JMI LOOP	11:	JMI 8	0110000010000000	6080
END:	MOV R3,T6	12:	MOV R3,T6	1100000001100110	C066
	STO T6,17	13:	STO T6,17	1101000100010000	D110
	HLT	14:	HLT	0000000000000000	0000
A:	4	15:	4	0000000000000100	0004
B:	3	16:	3	0000000000000011	0003
C:	0	17:	0	0000000000000000	0000

Figura 1: Ejemplo de programa para la CDB. Observa que el programa implementa la operación

$C = A \times B$, para $A \geq 0$ y $B \geq 0$. Por tanto, después de la ejecución, $C = 12\text{Dec} = 0C\text{Hex}$.



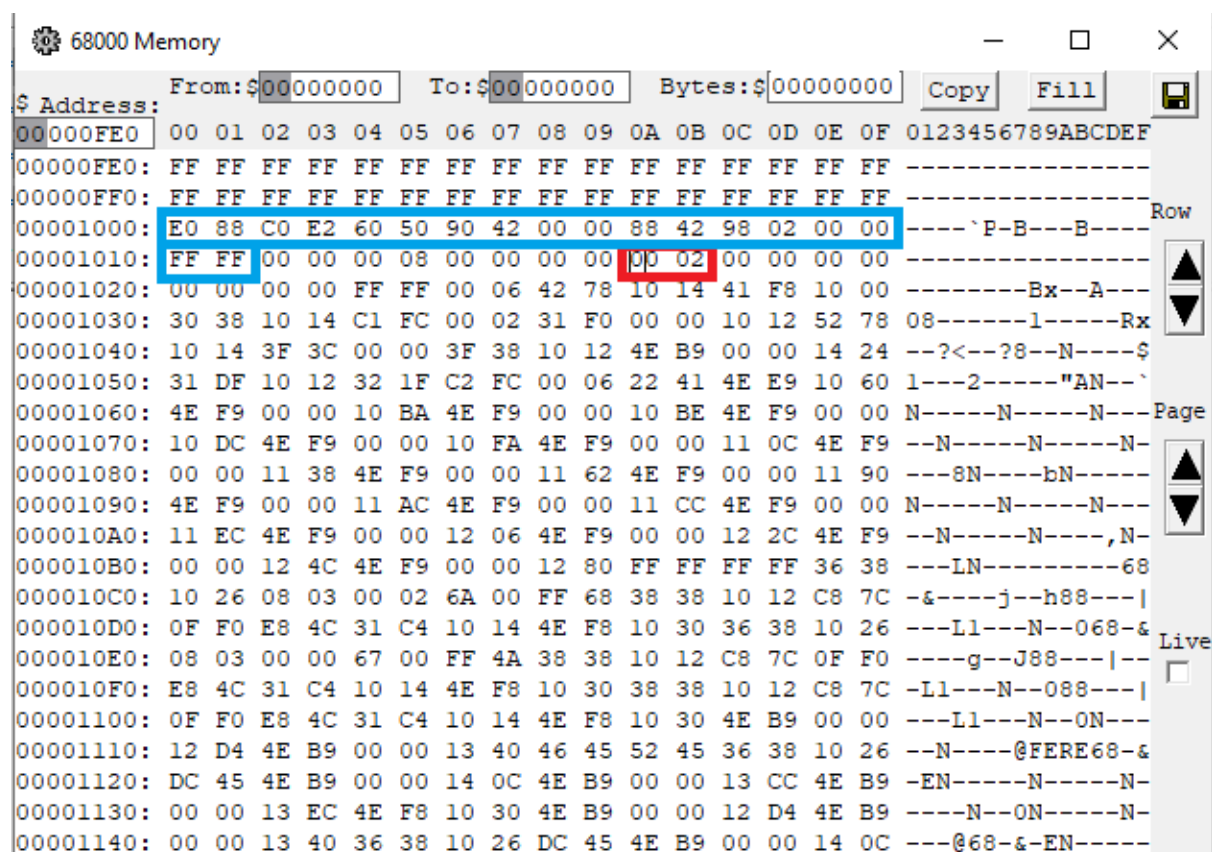
Observad la figura de arriba, en azul, está rodeado el EPROG completo y en rojo, el resultado.

Tras la ejecución del mismo, la posición de memoria del 68K correspondiente a @1022Hex deberá contener el valor 12Dec = 00CHex.

✓ **Ejemplo de programa número 2 para la CDB.**

Dirección @CDB	Ensamblador sin etiquetas	Instrucciones codificadas	Hex
0:	LOA 8,T7	1110000010001000	E088
1:	MOV T7,R2	1100000011100010	C0E2
2:	JMI 5	0110000001010000	6050
3:	SUB R2,R2	1001000001000010	9042
4:	HLT	0000000000000000	0000
5:	ADD R2,R2	1000100001000010	8842
6:	NEG R2	1001100000000010	9802
7:	HLT	0000000000000000	0000
8:	-1	1111111111111111	FFFF

Este programa realiza la operación $-1-1=-2$ y cambia el signo del resultado a 2.



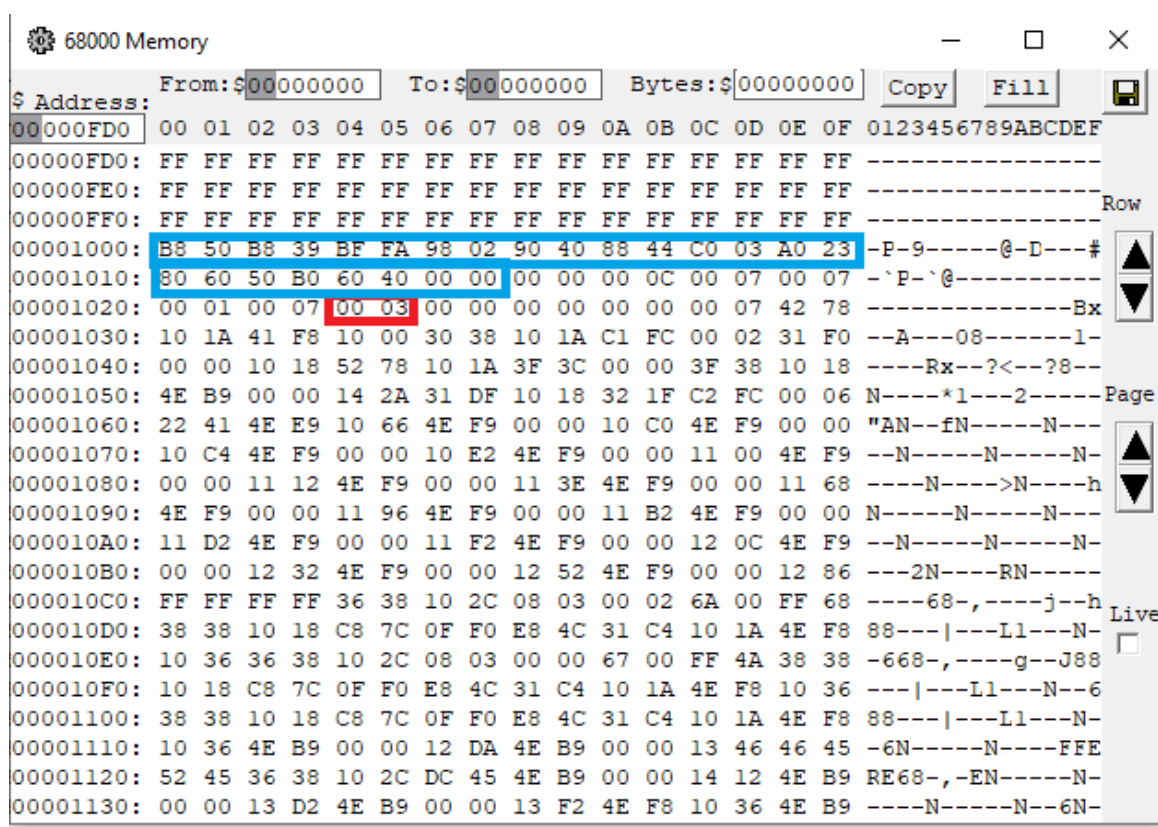
Observad la figura de arriba, en azul, está rodeado el EPROG completo y en rojo, el resultado.

Tras la ejecución del mismo, la posición de memoria del 68K correspondiente a ER2 (en este caso, @101AHex) deberá contener el valor 2Dec = 0002Hex.

✓ **Ejemplo de programa número 3 para la CDB.**

Dirección @CDB	Ensamblador sin etiquetas	Instrucciones codificadas	Hex
0:	SET.W #10,ER0	1011 1000 0101 0000	B850
1:	SET.W #7,ER1	1011 1000 0011 1001	B839
2:	SET.W #-1,ER2	1011 1111 1111 1010	BFFA
3:	NEG.W ER2	1001 1000 0000 0010	9802
4:	SUB.W ER2,ER0	1001 0000 0100 0000	9040
5:	ADD.W ER2,ER4	1000 1000 0100 0100	8844
6:	MOVE.W ER0,ER3	1100 0000 0000 0011	C003
7:	AND.W ER1,ER3	1010 0000 0010 0011	A023
8:	COM ER3,ER0	1000 0000 0110 0000	8060
9:	JMZ 11	0101 0000 1011 0000	50B0
10:	JMI 4	0110 0000 0100 0000	6040
11:	HALT	0000 0000 0000 0000	0000

Este programa realiza una cuenta atrás de ER0 hasta que iguale el valor de ER1, actualizando los flags mediante una AND de ambos. Se realizan un total de 3 iteraciones, de 10 hasta 7.



Tras la ejecución del mismo, la posición de memoria del 68K correspondiente a ER4 (en este caso, @1024Hex) deberá contener el valor 3Dec = 0003Hex.

6. Conclusiones

En conclusión, la elaboración de este minucioso proyecto, nos ha sido de gran utilidad, sobretodo por descubrir un micromundo, el cual ,a principio de semestre, no imaginábamos poder comprender. En nuestra opinión, creemos haber conseguido entender las profundidades de la informática, la complejidad de los procesadores. Además, hemos conseguido desarrollar una visión más analítica de cara a comprender programas escritos en lenguaje en ensamblador. Por último, comentar que nos ha parecido muy adecuada la metodología seguida para guiarnos en la realización de esta práctica.

Joan Balaguer, Jaume Adrover

7. Código fuente del programa

```
*-----
* Title      : PRAFIN21
* Written by : Jaume Adrover Fernandez, Joan Balaguer Llagostera
* Date       : 31/05/2021
* Description: Emulador de la CDB
*-----

    ORG $1000
EPROG:  DC.W $B803,$BFFC,$E0F0,$C0C0,$50C0,$E108,$C0E1,$50C0,$8803
        DC.W $8881,$50C0,$6080,$C066,$D110,$0000,$0004,$0003,$0000

        *Programa emulat 1: {EPROG:
*      *DC.W  $B803,$BFFC,$E0F0,$C0C0,$50C0,$E108,$C0E1,$50C0,$8803
        *DC.W $8881,$50C0,$6080,$C066,$D110,$0000,$0004,$0003,$0000}

        *Programa emulat 2: {EPROG:  DC.W
*      *$E088,$C0E2,$6050,$9042,$0000,$8842,$9802,$0000,$FFFF}

        *Programa emulat 3: {EPROG:  DC.W
*
$B850,$B839,$BFFA,$9802,$9040,$8844,$C003,$A023,$8060,$50A0,$6040,
*      *$0000}

EIR:    DC.W 0 ;eregistro de instruccion
EPC:    DC.W 0 ;econtador de programa
ER0:    DC.W 0 ;eregistro R0
ER1:    DC.W 0 ;eregistro R1
ER2:    DC.W 0 ;eregistro R2
ER3:    DC.W 0 ;eregistro R3
ER4:    DC.W 0 ;eregistro R4
ER5:    DC.W 0 ;eregistro R5
ET6:    DC.W 0 ;eregistro T6
ET7:    DC.W 0 ;eregistro T7
ESR:    DC.W 0 ;eregistro de estado (00000000 00000NCZ)

START:
    CLR.W EPC *Inicializamos a 0 el registro EPC
    LEA.L EPROG,A0
FETCH:
    ;--- IFETCH: INICIO FETCH
        ;*** En esta seccion debeis introducir el codigo necesario
para cargar
        ;*** en el EIR la siguiente instruccion a ejecutar,
indicada por el EPC
        ;*** y dejar listo el EPC para que apunte a la siguiente
instruccion
```

```

        ; ESCRIBID VUESTRO CODIGO AQUI
        MOVE.W EPC,D0
        MULS.W #2,D0          *Multiplicamos por 2 el valor del EPC
para obtener la
                                *dirección real a partir de la dirección
emulada
        MOVE.W X(A0,D0),EIR *Movemos la instrucción que debemos
realizar a EIR
        ADD.W #1,EPC
        ;--- FFETCH: FIN FETCH

        ;--- IBRDECOD: INICIO SALTO A DECOD
        ;*** En esta seccion debeis preparar la pila para llamar a
la subrutina
        ;*** DECOD, llamar a la subrutina, y vaciar la pila
correctamente,
        ;*** almacenando el resultado de la decodificación en D1

        ; ESCRIBID VUESTRO CODIGO AQUI
        MOVE.W #0,-(A7)      *Guardamos espacio en la pila para el
identificador
        MOVE.W EIR,-(A7)     *PUSH EIR

        JSR DECOD            *SALTO SUBROUTINA LIBRERÍA
        MOVE.W (A7)+,EIR     *POP EIR
        MOVE.W (A7)+,D1      *POP RESULTADO

        ;--- FBRDECOD: FIN SALTO A DECOD

        ;--- IBREXEC: INICIO SALTO A FASE DE EJECUCION
        ;*** Esta seccion se usa para saltar a la fase de
ejecucion
        ;*** NO HACE FALTA MODIFICARLA
        MULU #6,D1
        MOVEA.L D1,A1
        JMP JMPLIST(A1)
JMPLIST:
        JMP EHLT
        JMP EJMN
        JMP EJMZ
        JMP EJMI
        JMP ECOM
        JMP EADD
        JMP ESUB
        JMP ENEG
        JMP EAND
        JMP EOR
        JMP ENOT
        JMP ESET
        JMP EMOV

```

```

    JMP ESTO
    JMP ELOA
    ;--- FBREXEC: FIN SALTO A FASE DE EJECUCION

    ;--- IEXEC: INICIO EJECUCION
    ;*** En esta seccion debeis implementar la ejecucion de
    cada einstr.

    ; ESCRIBID EN CADA ETIQUETA LA FASE DE EJECUCION DE CADA
    INSTRUCCION
EHLT:
    SIMHALT

EJMN:
    MOVE.W ESR,D3
    BTST.L #2,D3      *Comprobamos qué valor tiene el tercer bit del
registro ESR
    BPL FETCH         *Si N es 0, volvemos a la fase de FETCH
    MOVE.W EIR,D4
    AND.W #$0FF0,D4   *Seleccionamos los bits de la dirección de
salto de la
                        *instrucción
    LSR.W #4,D4       *Movemos los bits seleccionados, 4 bits a la
derecha
    MOVE.W D4,EPC     *Modificamos el valor del EPC con la dirección
de salto

    JMP FETCH

EJMZ:
    MOVE.W ESR,D3
    BTST.L #0,D3      *Comprobamos qué valor tiene el primer bit del
registro ESR
    BEQ FETCH         *Si Z es 0, volvemos a la fase de FETCH
    MOVE.W EIR,D4
    AND.W #$0FF0,D4   *Seleccionamos los bits de la dirección de
salto de la
                        *instrucción
    LSR.W #4,D4       *Movemos los bits seleccionados, 4 bits a la
derecha
    MOVE.W D4,EPC     *Modificamos el valor del EPC con la dirección
de salto

    JMP FETCH

EJMI:
    MOVE.W EIR,D4
    AND.W #$0FF0,D4   *Seleccionamos los bits de la dirección de
salto de la
                        *instrucción

```

LSR.W #4,D4	*Movemos los bits seleccionados, 4 bits a la derecha
MOVE.W D4,EPC	*Modificamos el valor del EPC con la dirección de salto
JMP FETCH	*Volvemos a la fase de FETCH
ECOM:	
JSR REGISTERa	*Introducimos en D5 el valor del registro aaa
que nos	
	*indique la instrucción
JSR REGISTERb	*Introducimos en D6 el valor del registro bbb
que nos	
	*indique la instrucción
NOT.W D5	
ADD.W #1,D5	*Cambiamos el signo al segundo operando y le sumamos un 1
MOVE.W ESR,D3	
ADD.W D5,D6	*Realizamos una suma con el segundo operando modificado
JSR FLAGC	*Actualizamos el flag C segun el valor del
flag C del 68K	
JSR FLAGZ	*Actualizamos el flag Z segun el valor
contenido en D6	
JSR FLAGN	*Actualizamos el flag N segun el valor
contenido en D6	
JMP FETCH	*Volvemos a la fase de FETCH
EADD:	
JSR REGISTERa	*Introducimos en D5 el valor del registro aaa
que nos	
	*indique la instrucción
JSR REGISTERb	*Introducimos en D6 el valor del registro bbb
que nos	
	*indique la instrucción
MOVE.W ESR,D3	
ADD.W D5,D6	
JSR FLAGC	*Actualizamos el flag C segun el valor del
flag C del 68K	
MOVE.W D6,(A3)	*Guardamos resultado
JSR FLAGZ	*Actualizamos el flag Z segun el valor
contenido en D6	
JSR FLAGN	*Actualizamos el flag N segun el valor
contenido en D6	
JMP FETCH	*Volvemos a la fase de FETCH

ESUB:

JSR REGISTERa	*Introducimos en D5 el valor del registro aaa
que nos	
	*indique la instrucción
JSR REGISTERb	*Introducimos en D6 el valor del registro bbb
que nos	
	*indique la instrucción
NOT.W D5	
ADD.W #1,D5	*Cambiamos el signo al segundo operando y le
sumamos un 1	
MOVE.W ESR,D3	
ADD.W D5,D6	*Realizamos una suma con el segundo operando
modificado	
JSR FLAGC	
MOVE.W D6,(A3)	*Guardamos resultado
JSR FLAGZ	*Actualizamos el flag Z segun el valor
contenido en D6	
JSR FLAGN	*Actualizamos el flag N segun el valor
contenido en D6	
JMP FETCH	*Volvemos a la fase de FETCH
ENEG:	
JSR REGISTERb	*Introducimos en D6 el valor del registro bbb
que nos	
	*indique la instrucción
NOT.W D6	
ADD.W #1,D6	
MOVE.W D6,(A3)	*Guardamos resultado
JSR FLAGZ	*Actualizamos el flag Z segun el valor
contenido en D6	
JSR FLAGN	*Actualizamos el flag N segun el valor
contenido en D6	
JMP FETCH	*Volvemos a la fase de FETCH
EAND:	
JSR REGISTERa	*Introducimos en D5 el valor del registro aaa
que nos	
	*indique la instrucción
JSR REGISTERb	*Introducimos en D6 el valor del registro bbb
que nos	
	*indique la instrucción
AND.W D5,D6	
MOVE.W D6,(A3)	*Guardamos resultado
JSR FLAGZ	*Actualizamos el flag Z segun el valor
contenido en D6	
JSR FLAGN	*Actualizamos el flag N segun el valor
contenido en D6	
JMP FETCH	*Volvemos a la fase de FETCH

```

EOR:
    JSR REGISTERa    *Introducimos en D5 el valor del registro aaa
que nos
                    *indique la instrucción
    JSR REGISTERb    *Introducimos en D6 el valor del registro bbb
que nos
                    *indique la instrucción
    OR.W D5,D6
    MOVE.W D6,(A3)   *Guardamos resultado
    JSR FLAGZ        *Actualizamos el flag Z segun el valor
contenido en D6
    JSR FLAGN        *Actualizamos el flag N segun el valor
contenido en D6

    JMP FETCH        *Volvemos a la fase de FETCH

ENOT:
    JSR REGISTERb    *Introducimos en D6 el valor del registro bbb
que nos
                    *indique la instrucción
    NOT.W D6
    MOVE.W D6,(A3)   *Guardamos resultado
    JSR FLAGZ        *Actualizamos el flag Z segun el valor
contenido en D6
    JSR FLAGN        *Actualizamos el flag N segun el valor
contenido en D6

    JMP FETCH        *Volvemos a la fase de FETCH

ESET:
    JSR REGISTERb    *Introducimos en D6 el valor del registro bbb
que nos
                    *indique la instrucción
    MOVE.W EIR,D4
    AND.W #$07F8,D4 *Seleccionamos los bits kkkkkkkk que nos
indican el dato
                    *que introduciremos en el registro bbb
    LSR.W #3,D4      *Movemos dichos 8 bits, 3 posiciones a la
derecha
    EXT.W D4         *Extendemos el resultado de las anteriores
maniobras
                    *de 8 a 16 bits
    MOVE.W D4,D6
    MOVE.W D6,(A3)   *Guardamos resultado
    JSR FLAGZ        *Actualizamos el flag Z segun el valor
contenido en D6
    JSR FLAGN        *Actualizamos el flag N segun el valor
contenido en D6

```

```

        JMP FETCH          *Volvemos a la fase de FETCH

EMOV:
        JSR REGISTERa      *Introducimos en D5 el valor del registro aaa
que nos
                                *indique la instrucción
        JSR REGISTERb      *Introducimos en D6 el valor del registro bbb
que nos
                                *indique la instrucción
        MOVE.W D5,D6
        MOVE.W D6,(A3)     *Guardamos resultado
        JSR FLAGZ          *Actualizamos el flag Z segun el valor
contenido en D6
        JSR FLAGN          *Actualizamos el flag N segun el valor
contenido en D6

        JMP FETCH          *Volvemos a la fase de FETCH

ESTO:
        MOVE.W EIR,D4
        BTST.L #3,D4       *Miramos el valor del bit 3 de la instrucción
        BEQ TSEISSTO       *Decodificamos segun el valor del bit 3 de la
                                *instrucción
        *Caso T7 i=1
        AND.W #$0FF0,D4    *Seleccionamos los bits que nos indican la
                                *posición de memoria
        LSR.W #4,D4
        MULS.W #2,D4        *Multiplicamos la dirección donde queremos
meter el
                                *contenido de T7 por 2 ya que la dirección
emulada
                                *siempre es el doble de la dirección del 68K
        MOVE.W ET7,X(A0,D4) *Movemos el contenido del registro T7 a
la memoria

        JMP FETCH          *Volvemos a la fase de FETCH
TSEISSTO:
        *Caso T6 i=0
        AND.W #$0FF0,D4    *Seleccionamos los bits que nos indican la
                                *posición de memoria
        LSR.W #4,D4
        MULS.W #2,D4        *Multiplicamos la dirección donde queremos
meter el
                                *contenido de T6 por 2 ya que la dirección
emulada
                                *siempre es el doble de la dirección del 68K
        MOVE.W ET6,X(A0,D4) *Movemos el contenido del registro T6 a la
memoria

        JMP FETCH          *Volvemos a la fase de FETCH

```

ELOA:

```
    MOVE.W EIR,D4
    BTST.L #3,D4      *Miramos el valor del bit 3 de la instrucción
    BEQ TSEISLOA      *Decodificamos segun el valor del bit 3 de la
                        *instrucción

    *Caso T7
    AND.W #$0FF0,D4   *Seleccionamos los bits que nos indican la
                        *posición de memoria

    LSR.W #4,D4
    MULS.W #2,D4      *Multiplicamos la dirección donde queremos
meter el
                        *contenido de T7 por 2 ya que la dirección
emulada
                        *siempre és el doble de la dirección del 68K
    MOVE.W X(A0,D4),ET7 *Movemos el contenido de la posición de
                        *memoria dedel registro a T7

    MOVE.W ET7,D6
    JSR FLAGZ          *Actualizamos el flag Z segun el valor
contenido en D6
    JSR FLAGN          *Actualizamos el flag N segun el valor
contenido en D6

    JMP FETCH          *Volvemos a la fase de FETCH
TSEISLOA:
    *Caso T6
    AND.W #$0FF0,D4
    LSR.W #4,D4
    MULS.W #2,D4      *Multiplicamos la dirección donde queremos
meter el
                        *contenido de T6 por 2 ya que la dirección
emulada
                        *siempre és el doble de la dirección del 68K
    MOVE.W X(A0,D4),ET6 *Movemos el contenido de la posición de
                        *memoria dedel registro a T7

    MOVE.W ET6,D6
    JSR FLAGZ          *Actualizamos el flag Z segun el valor
contenido en D6
    JSR FLAGN          *Actualizamos el flag N segun el valor
contenido en D6

    JMP FETCH          *Volvemos a la fase de FETCH

;--- FEEXEC: FIN EJECUCION

;--- ISUBR: INICIO SUBROUTINAS
;*** Aqui debeis incluir las subrutinas que necesite
vuestra solucion
;*** SALVO DECOD, que va en la siguiente seccion
```

; ESCRIBID VUESTRO CODIGO AQUI

REGISTERa:

*Vamos decodificando en función de los valores de los bits
5, 6 y 7

*aaa de la instrucción que estemos realizando

MOVE.W EIR,D4

BTST.L #5,D4

BEQ PRIMERAa

BTST.L #6,D4

BEQ SEXTAa

BTST.L #7,D4

BEQ SEPTIMAa

MOVE.W ET7,D5 *Caso 111

RTS

PRIMERAa:

BTST.L #6,D4

BEQ TERCERAa

BTST.L #7,D4

BEQ CUARTAa

MOVE.W ET6,D5 *Caso 110

RTS

SEGUNDAa:

BTST.L #7,D4

BEQ CUARTAa

MOVE.W ER5,D5 *Caso 101

RTS

TERCERAa:

BTST.L #7,D4

BEQ QUINTAa

MOVE.W ER4,D5 *Caso 100

RTS

CUARTAa:

MOVE.W ER2,D5 *Caso 010

RTS

QUINTAa:

MOVE.W ER0,D5 *Caso 000

RTS

SEXTAa:

MOVE.W ER1,D5 *Caso 001

RTS

SEPTIMAa:

MOVE.W ER3,D5 *Caso 011

RTS

REGISTERb:

*Vamos decodificando en función de los valores de los bits
0, 1 y 2

*bbb de la instrucción que estemos realizando

```

        MOVE.W EIR,D4
        BTST.L #0,D4
        BEQ PRIMERAb
        BTST.L #1,D4
        BEQ SEXTAb
        BTST.L #2,D4
        BEQ SEPTIMAb
        MOVE.W ET7,D6 *Caso 111
        LEA.L ET7,A3 *Movemos la dirección efectiva del registro a
A3
        RTS
PRIMERAb:
        BTST.L #1,D4
        BEQ TERCERAb
        BTST.L #2,D4
        BEQ CUARTAb
        MOVE.W ET6,D6 *Caso 110
        LEA.L ET6,A3 *Movemos la dirección efectiva del registro a
A3
        RTS
SEGUNDAb:
        BTST.L #2,D4
        BEQ CUARTAb
        MOVE.W ER5,D6 *Caso 101
        LEA.L ER5,A3 *Movemos la dirección efectiva del registro a
A3
        RTS
TERCERAb:
        BTST.L #2,D4
        BEQ QUINTAb
        MOVE.W ER4,D6 *Caso 100
        LEA.L ER4,A3 *Movemos la dirección efectiva del registro a
A3
        RTS
CUARTAb:
        MOVE.W ER2,D6 *Caso 010
        LEA.L ER2,A3 *Movemos la dirección efectiva del registro a
A3
        RTS
QUINTAb:
        MOVE.W ER0,D6 *Caso 000
        LEA.L ER0,A3 *Movemos la dirección efectiva del registro a A3
        RTS
SEXTAb:
        MOVE.W ER1,D6 *Caso 001
        LEA.L ER1,A3 *Movemos la dirección efectiva del registro a A3
        RTS
SEPTIMAb:
        MOVE.W ER3,D6 *Caso 011
        LEA.L ER3,A3 *Movemos la dirección efectiva del registro a A3

```

RTS

FLAGZ:

```
MOVE.W ESR,D3
CMP #0,D6
BNE FLAGZCLR *Miramos si el resultado es igual a 0 o no
BSET.L #0,D3 *Ponemos el flag Z a 1
MOVE.W D3,ESR
RTS
```

FLAGZCLR:

```
BCLR.L #0,D3 *Ponemos el flag Z a 0
MOVE.W D3,ESR
RTS
```

FLAGN:

```
MOVE.W ESR,D3
CMP #0,D6
BMI FLAGNCLR *Miramos a ver si el resultado es negativo o no
BSET.L #2,D3 *Ponemos el flag N a 1
MOVE.W D3,ESR
RTS
```

FLAGNCLR:

```
BCLR #2,D3 *Ponemos el flag N a 0
MOVE.W D3,ESR
RTS
```

FLAGC:

```
BCC FLAGCCLR *Miramos el valor actual del flag N del 68K
BSET.L #1,D3 *ponemos el flag C=1
MOVE.W D3,ESR
RTS
```

FLAGCCLR:

```
BCLR #2,D3 Ponemos el flag C=0
MOVE.W D3,ESR
RTS
```

;--- FSUBR: FIN SUBROUTINAS

;--- IDECOD: INICIO DECOD

```
;*** Tras la etiqueta DECOD, debeis implementar la
subrutina de
;*** decodificacion, que debera ser de libreria, siguiendo
la interfaz
;*** especificada en el enunciado
```

DECOD:

```

        ; ESCRIBID VUESTRO CODIGO AQUI
MOVE.L D0,-(A7)  *PUSH D0 A LA PILA
MOVE.W 8(A7),D0  *MOVEMOS EIR A D0

BIT1:

BTST #15,D0
BNE  SEGUNDA_RAMAS *Realizaremos el salto
                        *si el bit 15 es un 1

*CASOS (0-3)-----BIT_1=0

PRIMERA_RAMAS:  *Caso (0,...)
BTST #14,D0
BNE  PRIMERA_RAMAS_PRIMER_NIVEL_UNO *Realizaremos el salto
                        *si el bit 14 es un 1

*DECODIFICAR/ALMACENAR CASO 0,0=HALT
MOVE.W #0,10(A7)  *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

*Casos (1-3)
PRIMERA_RAMAS_PRIMER_NIVEL_UNO:  *Caso(0,1)
BTST #13,D0
BEQ  PRIMERA_RAMAS_SEGUNDO_NIVEL_CERO  *Realizaremos el salto
                        *si el bit 13 es un 0

        *Caso (0,1,1)-----Caso 3
        *DECODIFICAR/ALMACENAR CASO 0,1,1
MOVE.W #3,10(A7)  *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

PRIMERA_RAMAS_SEGUNDO_NIVEL_CERO: *(0,1,0)
BTST #12,D0
BNE  PRIMERA_RAMAS_TERCER_NIVEL_UNO *Realizaremos el salto
                        *si el bit 12 es un 1

PRIMERA_RAMAS_TERCER_NIVEL_CERO: *(0,1,0,0)
        *DECODIFICAR/ALMACENAR CASO (0,1,0,0)----Caso 1
MOVE.W #1,10(A7)  *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

PRIMERA_RAMAS_TERCER_NIVEL_UNO: *(0,1,0,1)

```



```

    *ALMACENAR CASO (0,1,0,1)----Caso 2
MOVE.W #2,10(A7)    *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

*CASOS (4-14)----BIT_1=1

SEGUNDA_RAMA:      *(1,n)
BTST #14,D0
BEQ  SEGUNDA_RAMA_PRIMER_NIVEL_CERO    *Realizaremos el salto
                                         *si el bit 14 es 0

*Casos (12-14)
SEGUNDA_RAMA_PRIMER_NIVEL_UNO: *(1,1)
BTST #13,D0
BEQ  SEGUNDA_RAMA_SEGUNDO_NIVEL_CERO

SEGUNDA_RAMA_SEGUNDO_NIVEL_UNO:
*DECODIFICAR/ALMACENAR Caso(1,1,1)---14
MOVE.W #14,10(A7)    *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

SEGUNDA_RAMA_SEGUNDO_NIVEL_CERO:      *(1,1,0)
BTST #12,D0
BNE  SEGUNDA_RAMA_TERCER_NIVEL_UNO    *Realizamos el salto
                                         *si el bit 12 es un 1

SEGUNDA_RAMA_TERCER_NIVEL_CERO: *(1,1,0,0)
*DECODIFICAR/ALMACENAR Caso(1,1,0,0)---12
MOVE.W #12,10(A7)    *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

SEGUNDA_RAMA_TERCER_NIVEL_UNO: *(1,1,0,1)
*DECODIFICAR/ALMACENAR Caso(1,1,0,1)-----13
MOVE.W #13,10(A7)    *Guardamos decodificación en resultado
MOVE.L (A7)+,D0
RTS

*Casos (4-11)
SEGUNDA_RAMA_PRIMER_NIVEL_CERO: *(1,0)
BTST #13,D0
BNE  SEGUNDA_RAMA_SEGUNDO_NIVEL_UNOO    *Realizaremos el salto
                                         *si el bit 13 es 1

```

```

*Casos (4-7)
SEGUNDA_RAMAS_SEGUNDO_NIVEL_CEROO:    *(1,0,0)
    BTST #12,D0
    BNE SEGUNDA_RAMAS_TERCER_NIVEL_UNOO  *Realizaremos el salto
                                           *si el bit 12 es 1

*Casos (4,5)
SEGUNDA_RAMAS_TERCER_NIVEL_CEROO:    *(1,0,0,0)
    BTST #11,D0
    BNE  SEGUNDA_RAMAS_CUARTO_NIVEL_UNO   *Realizaremos el salto
                                           *si el bit 11 es 1

*Caso 4
SEGUNDA_RAMAS_CUARTO_NIVEL_CERO:    *(1,0,0,0,0)
    *DECODIFICAR/ALMACENAR CASO 4
    MOVE.W #4,10(A7)  *Guardamos decodificación en resultado
    MOVE.L (A7)+,D0
    RTS

*Caso 5
SEGUNDA_RAMAS_CUARTO_NIVEL_UNO:    *(1,0,0,0,1)----Caso 5
    *DECODIFICAR/ALMACENAR CASO 5
    MOVE.W #5,10(A7)  *Guardamos decodificación en resultado
    MOVE.L (A7)+,D0
    RTS

*Casos (6,7)
SEGUNDA_RAMAS_TERCER_NIVEL_UNOO:    *(1,0,0,1)
    BTST #11,D0
    BNE  SEGUNDA_RAMAS_CUARTO_NIVEL_UNOO  *Realizaremos el salto
                                           *si el bit 11 es 1

SEGUNDA_RAMAS_CUARTO_NIVEL_CEROO:    *(1,0,0,1,0)
    *DECODIFICAR/ALMACENAR CASO 6
    MOVE.W #6,10(A7)  *Guardamos decodificación en resultado
    MOVE.L (A7)+,D0
    RTS

SEGUNDA_RAMAS_CUARTO_NIVEL_UNOO:    *(1,0,0,1,1)
    *DECODIFICAR/ALMACENAR CASO 7
    MOVE.W #7,10(A7)  *Guardamos decodificación en resultado
    MOVE.L (A7)+,D0
    RTS

*Casos (8-11)
SEGUNDA_RAMAS_SEGUNDO_NIVEL_UNOO:    *(1,0,1)
    BTST #12,D0
    BNE SEGUNDA_RAMAS_TERCER_NIVELUNO

*Casos (8,9)
SEGUNDA_RAMAS_TERCER_NIVELCERO:    *(1,0,1,0)
    BTST #11,D0

```

```

        BNE SEGUNDA_RAMAS_CUARTO_NIVELUNO

SEGUNDA_RAMAS_CUARTO_NIVELCERO: *(1,0,1,0,0)---Caso 8
        *DECODIFICAR/ALMACENAR CASO 8
        MOVE.W #8,10(A7) *Guardamos decodificación en resultado
        MOVE.L (A7)+,D0
        RTS

SEGUNDA_RAMAS_CUARTO_NIVELUNO: *(1,0,1,0,1)----Caso 9
        *DECODIFICAR/ALMACENAR CASO 9
        MOVE.W #9,10(A7) *Guardamos decodificación en resultado
        MOVE.L (A7)+,D0
        RTS

*Casos (10,11)
SEGUNDA_RAMAS_TERCER_NIVELUNO:          *(1,0,1,1)
        BTST #11,D0
        BNE SEGUNDA_RAMAS_CUARTO_NIVEL_UNOOO

SEGUNDA_RAMAS_CUARTO_NIVEL_ZERO:
        *DECODIFICAR/ALMACENAR CASO 10
        MOVE.W #10,10(A7) *Guardamos decodificación en resultado
        MOVE.L (A7)+,D0
        RTS

SEGUNDA_RAMAS_CUARTO_NIVEL_UNOOO:
        *DECODIFICAR/ALMACENAR CASO 11
        MOVE.W #11,10(A7) *Guardamos decodificación en resultado
        MOVE.L (A7)+,D0
        RTS

        ;--- FDECOD: FIN DECOD
FINAL:          *PROVISIONAL FINAL
        END          START

```

